PHASE II PROJECT DOCUMENTATION

REAL-TIME COLLABORATIVE TASK BOARD

TaskBoard | Native Android Application built with Kotlin, XML and Firebase

Course	SEC042 – New Age Programming Languages Lab
Student Name	Vipul , Vansh
Roll no	2401350017 , 2401350002
University	K.R. Mangalam University
Project Type	Native Android Application (Kotlin, XML, MVVM, Firebase)
Submission	Phase II – Capstone Project
Submission Date	10 November 2026
GitHub Repository	viipulll/TaskBoard

Contents: project overview, objectives, features, technologies, architecture, data model, implementation status, testing, repository and future scope.

1. Project Overview

TaskBoard is a native Android application that helps individuals and small teams plan and track work on a shared task board. It follows a Kanban-style workflow in which every task sits in a column that reflects its status: To Do, In Progress or Done. Moving a task across columns gives the whole team an immediate picture of what is pending, what is being worked on and what is finished.

The application is developed in Kotlin using XML-based layouts in Android Studio. It is structured with the MVVM architecture so that the user interface, screen state and data access remain separate and easy to maintain. Firebase Authentication and Cloud Firestore form the cloud layer, providing user accounts and a shared, real-time data store so that changes made by one team member can appear on the devices of the others.

> **In short:** TaskBoard replaces scattered chats, notes and spreadsheets with one mobile workspace where tasks are organised by status and visible to the whole team.

2. Problem Statement and Motivation

Students and small project teams commonly coordinate work through group chats, personal notes and separate spreadsheets. This creates several recurring problems:

- **Unclear ownership:** it is hard to tell who is responsible for which task.
- **Hidden progress:** the current state of work is buried in message history or out-of-date files.
- **Lack of a common view:** each member keeps a different picture of what is pending, so effort is duplicated or tasks are missed.
- **Poor mobile experience:** spreadsheets and desktop tools are awkward to update quickly from a phone.

TaskBoard addresses these problems by giving a team a single, mobile-first board. Tasks are grouped by status, each task carries its own details, and the shared data store keeps every collaborator looking at the same information.

3. Objectives

- Develop a native Android task-management application using Kotlin.
- Provide a clean, intuitive XML-based interface for navigating boards and tasks.
- Organise work with a Kanban-style workflow and clear task statuses.
- Support secure user accounts through Firebase Authentication (email and password).
- Use Cloud Firestore as the shared data store for collaborative task updates.
- Follow the MVVM architecture to separate UI, state handling and data operations.
- Validate the application on an Android emulator and a physical Android device.

4. Key Features

The table below lists the main features of TaskBoard and the purpose of each.

Feature	Description	Status
User registration and login	Email-and-password sign-up and sign-in through Firebase Authentication, with input validation and error messages.	In development
Dashboard	Home screen that summarises boards and recent task activity and acts as the entry point to the rest of the app.	In development
Kanban board	Tasks arranged in To Do, In Progress and Done columns, displayed as cards in RecyclerView lists.	In development
Task creation and details	Form to create a task (title, description, status) and a detail view to review or edit it.	In development
Team workspace	Shared workspace in which several members work on the same board and see the same tasks.	Planned
Activity and profile	Activity screen showing recent changes and a profile screen with account details and logout.	In development
Real-time synchronisation	Firestore snapshot listeners that push changes to all connected devices without a manual refresh.	Planned

> **Status key:** “In development” means the screens and code have been written but are not yet running end to end. “Planned” means the feature is part of the design and is still to be implemented. Statuses reflect the state of the project at the time of submission.

5. Technologies Used

Technology	Purpose in TaskBoard
Kotlin	Primary language for all application, ViewModel and repository logic.
Android Studio	IDE used to write, build, debug and test the application; Gradle Kotlin DSL for builds.
XML layouts	Define every screen and UI component using Material-style widgets.
ViewBinding	Type-safe access to views, avoiding findViewById errors and null-pointer crashes.
RecyclerView	Efficient scrolling lists for task cards and board columns.
MVVM + ViewModel	Keeps UI state out of Activities/Fragments so it survives configuration changes such as screen rotation.
Firebase Authentication	Email-and-password registration, login and logout.
Cloud Firestore	Cloud NoSQL database for user profiles, boards and tasks, with real-time listeners.
Kotlin Coroutines / Flow	Asynchronous work and observable data streams between repository and ViewModel.
Git and GitHub	Version control and hosting of the project source code.

6. System Architecture and Design

TaskBoard follows the Model–View–ViewModel (MVVM) pattern. Each layer has one responsibility and talks only to the layer directly beneath it.

View layer	XML layouts • Activities / Fragments • RecyclerView adapters. Displays state and forwards user actions.
▼	
ViewModel layer	Screen state • UI events • Kotlin Coroutines / Flow. Holds logic that is independent of the UI.
▼	
Data layer	Repositories • Firebase Authentication • Cloud Firestore. Single source of truth for data.

Data flow

1. The user performs an action in the View, for example tapping “Add task”.
2. The Activity/Fragment passes the action to the relevant ViewModel.
3. The ViewModel calls a repository function inside a coroutine.
4. The repository reads from or writes to Firebase Authentication or Cloud Firestore.
5. The result is exposed as state (Flow/LiveData) and the View updates automatically.

Code organisation

The source code is divided into packages so that each concern is easy to locate:

Package	Contents
ui	Activities, Fragments and RecyclerView adapters for each screen.
viewmodel	ViewModel classes holding and exposing screen state.
repository	Classes that wrap Firebase Authentication and Firestore access.
model	Data classes such as User, Board and Task.
navigation	Navigation graph and destinations between screens.
utils	Helpers for validation, formatting and constants.

Design decisions

- **MVVM:** chosen so the UI stays thin and logic can be changed or tested without touching layouts.
- **Cloud Firestore:** chosen because its snapshot listeners deliver real-time updates without building a custom backend.
- **Native Kotlin + XML:** gives full access to Android features and predictable performance for list-heavy screens.

7. Data Model

Cloud Firestore stores data as collections of documents. The planned structure for TaskBoard is shown below.

Collection	Key fields	Description
users	uid, name, email	One document per registered user, keyed by the Firebase Authentication UID.
boards	boardId, name, ownerId, memberIds	A shared board and the list of users who can access it.
tasks	taskId, boardId, title, description, status, createdBy, createdAt	A task card; the status field (TODO, IN_PROGRESS, DONE) decides which column shows it.

A snapshot listener on the tasks of the current board lets every member's device receive new, edited and moved tasks as they happen.

8. Implementation Description

1. **Project foundation:** set up the Android project in Android Studio with Kotlin, XML layouts, ViewBinding and Gradle Kotlin DSL.
2. **User interface (in development):** building the main navigation and the screens for authentication, dashboard, Kanban board, task form and details, activity and profile. Several screens are completed in code.
3. **Application structure:** organise the code into UI, ViewModel, repository, model, navigation and utility packages following MVVM.
4. **Authentication integration (planned):** connect Firebase Authentication to the registration, login and logout flows with validation of email and password.
5. **Cloud data integration (planned):** store user and task data in Cloud Firestore and attach real-time listeners for shared board updates.
6. **Validation and testing (planned):** verify navigation, input validation, authentication, task operations and synchronisation on an emulator and a physical device.

Implementation status

Area	Status	Notes
Project setup and architecture	Completed	Packages, ViewBinding and MVVM structure in place.
User interface screens	In development	Several screens completed in code; not yet running end to end.
Firebase Authentication	Planned	Registration, login and logout integration.
Cloud Firestore data layer	Planned	Task storage and real-time listeners.
Testing on emulator and device	Not started	Test plan in Section 9; to be run once the app builds and runs.

> **Note:** The application is not yet running end to end. Test results (Section 9) will be recorded once it runs.

9. Testing and Validation

The application will be tested on an Android emulator and a physical Android device. The planned test cases are listed below; results will be recorded after testing.

No.	Test case	Expected result	Result
1	Register with a valid email and password	Account is created and the user reaches the dashboard.	Pending
2	Register or log in with an invalid email or short password	Validation message is shown; no request is sent.	Pending
3	Log in with correct credentials	User is signed in and the dashboard is displayed.	Pending
4	Log out from the profile screen	Session ends and the login screen is shown.	Pending
5	Create a task with a title	Task appears in the To Do column.	Pending
6	Move a task between columns	Task status updates and it shows in the new column.	Pending
7	Rotate the screen on the board	State is preserved; no crash or data loss.	Pending
8	Change a task on one device	Change appears on a second signed-in device.	Pending

10. Repository

GitHub repository: [viipulll/TaskBoard]

The repository contains the Android Studio project, source code and resources, together with a README that describes the project and how to run it:

- Prerequisites: Android Studio, a recent JDK and an emulator or Android device.
- Setup: clone the repository, add your own google-services.json from a Firebase project, and sync Gradle.
- Run: select a device and press Run in Android Studio.

Private keys and sensitive configuration files are not committed to the repository.

11. Conclusion and Future Scope

TaskBoard provides a straightforward mobile workspace for organising and tracking team tasks. The Kotlin and XML implementation with MVVM gives the project a clear, maintainable structure, and the planned Firebase layer will support authenticated users and shared real-time task data. The project applies native Android development, architectural separation of concerns and cloud services for collaboration.

Possible future improvements include:

- Task assignment to specific team members and due dates.
- Push notifications for new or changed tasks.

- Member roles and permissions (owner, editor, viewer).
- Offline support with automatic synchronisation.
- Richer activity history, search and filtering.

12. References

- Android Developers. Android documentation. https://developer.android.com/
- Kotlin Documentation. https://kotlinlang.org/docs/home.html
- Firebase Documentation. https://firebase.google.com/docs
- Cloud Firestore Documentation. https://firebase.google.com/docs/firestore
- Android Developers. Guide to app architecture. https://developer.android.com/topic/architecture